Engineer Internship Roadmap

30-Day Comprehensive Program

Xeven Solutions

NLP & LangChain Specialization Track

Program Overview

This comprehensive 30-day intensive internship program is designed to transform beginners into capable AI Engineers with specialized expertise in Natural Language Processing (NLP), LangChain, and modern API development. The program emphasizes hands-on practical skills over theoretical knowledge, with a 70% practical to 30% theoretical ratio. The extended timeline allows for deeper understanding, more practice time, and development of both technical and soft skills essential for professional software development teams.

Key Learning Objectives

- Build production-ready NLP applications using LangChain and modern LLMs
- Develop RESTful APIs with FastAPI and PostgreSQL integration
- Implement RAG (Retrieval-Augmented Generation) systems for intelligent document processing
- Master Python best practices following PEP 8 coding standards
- Maintain professional GitHub profiles with daily code commits and proper documentation
- Develop research skills by consulting multiple AI sources (ChatGPT, Gemini, Claude) and technical articles
- Learn professional ethics, team collaboration, and effective technical communication
- Build soft skills: code reviews, documentation, mentoring, and constructive feedback

Daily Structure & Requirements

Daily Schedule

Time	Activity
Morning (2-3 hrs)	Theoretical Learning (30%) - Research from 3 AI sources + 1 article
Afternoon (4-5 hrs)	Practical Implementation (70%) - Coding, building projects, debugging in Jupyter notebooks
End of Day (1 hr)	Documentation, GitHub commit, and email progress report to assigned lead

Mandatory Research Process (Every Concept)

For EVERY theoretical concept, you MUST :

- Consult ChatGPT to explain the concept with examples
- Consult Google Gemini for technical details and use cases
- Consult Claude (claude.ai) for practical implementation tips
- Read at least ONE Medium/Dev.to/technical blog article on the topic
- Compare all 4 sources - create comparison table in your notebook
- Synthesize information into your own understanding
- Document ALL sources with dates and URLs in References section
- Note which source provided the clearest explanation and why

Example Research Documentation Format:

References for RAG Concept (Day 18)

1. ChatGPT (Consulted: January 15, 2026)

- Explained RAG as combining retrieval with generation

- Provided analogy: "Like having a smart assistant with access to a library"

2. Google Gemini (Consulted: January 15, 2026)

- Provided technical architecture diagrams

- Explained vector similarity search process

3. Claude (Consulted: January 15, 2026)

- Shared LangChain implementation code examples

- Explained best practices for chunk sizes

4. Medium Article: "Understanding RAG Systems" by John Doe

- URL: https://medium.com/article-link

- Published: January 10, 2026

- Key insight: Importance of semantic chunking for better retrieval

Comparison: All sources agreed RAG improves accuracy for domain-specific tasks.

Clearest Explanation: Claude's code examples made implementation most concrete.

Required Daily Deliverables

1. Jupyter Notebook (.ipynb) with:

- Clear markdown cells explaining each concept researched
- Code cells with inline comments following PEP 8 standards
- Output cells showing execution results
- References section citing all 3 AI sources + 1 article with dates
- Error handling and debugging notes if issues encountered
- Summary section with key learnings and insights

2. GitHub Repository:

- Organized folder structure (e.g., /day01, /day02, etc.)
- Descriptive commit messages (e.g., 'Day 5: Built ML decision system with research notes')
- Updated README.md with daily progress summary
- requirements.txt file with all dependencies and versions
- LEARNINGS.md file documenting research insights and comparisons

3. Email Progress Report:

- Subject: '[Day X] Your Name - Daily Progress Report'
- What was learned (theoretical concepts with brief explanation)
- What was built (practical implementations with outcomes)
- Research sources consulted (list all 4 sources with key takeaways)
- Challenges faced and how they were resolved
- GitHub commit link
- One professional insight or soft skill learned today

- Questions or blockers for the next day

PEP 8 Coding Standards (Mandatory)

- Use 4 spaces for indentation (never tabs)
- Maximum line length: 79 characters for code, 72 for docstrings
- Variable names: lowercase_with_underscores (snake_case)
- Function names: lowercase_with_underscores
- Class names: CapitalizedWords (PascalCase)
- Constants: UPPERCASE_WITH_UNDERSCORES
- Two blank lines between top-level functions and classes
- One blank line between methods inside a class
- Always include docstrings for functions and classes with description, parameters, and return values

GitHub Profile Maintenance

- Professional profile picture and bio describing your role as AI Engineer Intern
- Green contribution graph with daily commits throughout the 30 days
- Repository naming convention: ai-internship-xeven-2026
- Comprehensive README.md with project overview, setup instructions, and daily logs
- .gitignore file to exclude sensitive data, API keys, and cache files (__pycache__, .env)

Professional Ethics & Team Collaboration

Code of Professional Conduct

Being a great developer is not just about writing code. Professional ethics, integrity, and effective communication are equally important for career success and team productivity. These principles will guide you throughout your career.

Core Professional Ethics:

- **Honesty & Integrity:** Never plagiarize code. Always credit sources, libraries, and tutorials used. If you find a solution on Stack Overflow, cite it in comments
- **Intellectual Property:** Respect software licenses (MIT, GPL, Apache). Never steal or redistribute proprietary code without permission
- **Code Quality:** Write clean, maintainable code that others can understand and extend. Your code will be read more than written
- **Security Awareness:** NEVER commit API keys, passwords, database credentials, or sensitive data to GitHub. Use environment variables (.env files)
- **Continuous Learning:** Technology evolves rapidly. Stay humble, embrace new tools and techniques, and never stop learning

- **Admit Mistakes:** Everyone makes errors. Own your mistakes, learn from them, document solutions, and help others avoid the same issues
- **Time Management:** Meet deadlines and commitments. If you're blocked or falling behind, communicate early (not at the last minute)
- **Respect Privacy:** Never access, share, or misuse user data without proper authorization. Treat all data with confidentiality

Effective Team Communication

Software development is a team sport. Your ability to communicate clearly and collaborate effectively will determine your success as much as your technical skills. These practices are used by professional developers worldwide.

- **Ask Smart Questions:** Research first (Google, documentation, Stack Overflow), then ask specific questions with context. Include: what you tried, what you expected, what actually happened, relevant code snippets
- **Document Everything:** Write clear README files, meaningful code comments, and descriptive commit messages. Future you (and your teammates) will thank you
- **Give Constructive Feedback:** When reviewing code, be kind but honest. Suggest improvements with explanations: 'Consider using list comprehension here for better readability' vs 'This code is bad'
- **Receive Feedback Gracefully:** Don't take code reviews personally. Every critique is a learning opportunity. Ask questions if feedback is unclear
- **Daily Standups:** Share progress, blockers, and plans concisely (2-3 minutes). Focus on what helps the team, not lengthy explanations
- **Asynchronous Communication:** Write clear Slack/email messages. Use bullet points for complex information. Respect time zones in global teams
- **Active Listening:** In meetings and discussions, listen more than you speak. Understand the full context before responding or offering solutions
- **Conflict Resolution:** Disagree professionally. Focus on technical merits and data, not personal attacks. Say 'I think approach A because...' not 'You're wrong'
- **Mentorship:** Help junior developers when possible. Teaching others reinforces your own learning and builds team culture
- **Cultural Sensitivity:** Respect diverse backgrounds, time zones, work styles, and communication preferences in global teams

Code Review Best Practices

Code reviews are essential for maintaining code quality, sharing knowledge, and preventing bugs. Approach them as collaborative learning opportunities, not criticism sessions.

When Your Code is Being Reviewed:

- Provide context in the pull request description: What changed? Why? What problem does it solve?
- Keep pull requests small and focused (< 400 lines when possible). Easier to review = faster approval
- Test your code thoroughly before requesting review. Run all tests, check edge cases
- Respond to feedback promptly and professionally within 24 hours
- Ask clarifying questions if feedback is unclear: 'Could you explain what you mean by...'
- Don't argue defensively - assume good intent. Reviewers want to help you improve
- Thank reviewers for their time and insights

When Reviewing Others' Code:

- Start with positive feedback - acknowledge good work, clever solutions, or clean code
- Be specific with suggestions: 'Consider using list comprehension here for readability' vs 'This code is messy'
- Suggest, don't command: 'What do you think about...' vs 'You must change this'
- Explain the 'why' behind your suggestions: 'Using enumerate() here improves readability because...'
- Distinguish between critical issues (bugs, security flaws) and nitpicks (style preferences)
- Approve when it's good enough, not perfect. Don't block on minor style issues
- Review within 24 hours. Delayed reviews block progress and demotivate teammates

Time Management & Productivity

Effective time management is crucial for completing this intensive 30-day program while maintaining quality work and preventing burnout. These techniques are used by professional developers worldwide.

- **Pomodoro Technique:** Work in 25-minute focused blocks with 5-minute breaks. After 4 pomodoros, take a 15-30 minute break. Prevents burnout
- **Deep Work Sessions:** Block 2-3 hours for complex coding without interruptions. Turn off notifications, close Slack, focus on one hard problem
- **Avoid Multitasking:** Context switching kills productivity. Focus on one task at a time until it's complete or you hit a blocker
- **Learn to Say No:** Don't overcommit to extra tasks during the internship. Deliver quality work on deadlines rather than rush everything
- **Take Breaks:** Your brain needs rest. Walk, stretch, or meditate between coding sessions. Stuck on a bug? Step away for 10 minutes
- **Track Your Time:** Use tools like Toggl or RescueTime to understand where your time goes. Optimize based on data, not feelings
- **Energy Management:** Do hard problems when you're most alert (morning for most people). Save routine tasks for low-energy times
- **Automate Repetitive Tasks:** If you do something more than twice, write a script for it. Saves time and prevents errors

Weekly Soft Skills Practice

Throughout the 30-day program, you will actively practice these soft skills:

- **Week 1 (Days 1-7):** Documentation - Write clear README files, comprehensive code comments, and detailed commit messages
- **Week 2 (Days 8-14):** Code Reviews - Review a peer's code and receive feedback on yours. Practice giving constructive feedback
- **Week 3 (Days 15-21):** Technical Presentation - Present your RAG project to the team. Practice explaining complex concepts simply
- **Week 4 (Days 22-28):** Mentoring - Help a junior intern or peer with a blocker they're facing. Teaching reinforces your learning
- **Ongoing (Daily):** Daily Standups - Share progress in brief, clear updates. Practice concise technical communication

30-Day Learning Path

Week 1: Python Foundations & AI Fundamentals

Day 1: AI Fundamentals & Environment Setup

Morning Session: Theoretical Concepts (2-3 hours)

Research Requirements: Consult ChatGPT + Gemini + Claude + 1 article for each concept

- What is Artificial Intelligence? Definition, history, and real-world impact
- Types of AI: Narrow AI (current) vs. General AI (future conceptual understanding)
- AI Domains and applications: Healthcare (diagnosis), Finance (fraud detection), Education (personalized learning), Marketing (recommendations)
- Fields of AI: Machine Learning, Deep Learning, AI Engineering roles and responsibilities
- Introduction to NLP (text understanding), Speech Recognition (voice), Computer Vision (images)
- Role of an AI Engineer in 2026 : Building, deploying, and maintaining AI systems
- Difference between Training (learning from data) vs. Inference (making predictions)

Afternoon Session: Practical Implementation (4-5 hours)

- Download and install Python 3.11+ from python.org (verify with python --version)
- Install VS Code from code.visualstudio.com and add Python extension
- Understanding virtual environments vs. global environments (isolation and dependency management)
- Install and set up UV (modern Python package installer): curl -LsSf https://astral.sh/uv/install.sh | sh
- Create project folder structure: ai-internship-xeven-2026/day01/
- Write first Python script: app.py with personalized greeting using print()
- Create GitHub account (if needed) and configure Git with name and email
- Initialize Git repository, create .gitignore, and make first commit
- Write comprehensive README.md explaining the project

Practical Tasks

Task 1: Environment Setup & First Script

- Create app.py with: print('Hi, this is [Your Name], future AI engineer!')
- Run the script in VS Code terminal and capture screenshot
- Document the entire setup process in a Jupyter notebook with markdown explanations
- Include: installation steps, screenshots, troubleshooting notes

Task 2: Research Task (MANDATORY)

- Research 'What is Artificial Intelligence?' from ChatGPT, Gemini, and Claude
- Ask each: 'Explain what Artificial Intelligence is with real-world examples'
- Read one Medium article on 'Introduction to Artificial Intelligence'
- Create comparison table showing: Source, Key Points, Examples Provided, Clarity Rating
- Document all 4 sources with dates and URLs in References section

Task 3: Communication Task

- Write 200-word introduction: 'What is AI and the Role of an AI Engineer'
- Write for complete beginners - avoid technical jargon
- Include: definition, real examples, why it matters, what AI engineers do
- Add to GitHub README.md with proper markdown formatting (headers, lists)

Expected Deliverables

- Functional Python environment with UV configured and tested
- GitHub repository created with first commit and professional README.md
- Jupyter notebook documenting setup steps with screenshots
- LEARNINGS.md file with research sources cited (4 sources with comparison)
- Email progress report sent to lead by end of day with all links

Soft Skill Focus : Documentation : Practice writing clear, beginner-friendly technical explanations

Day 2: Python Basics - Variables & Data Types

Morning Session: Theoretical Concepts (2-3 hours)

Research Requirements: Consult ChatGPT + Gemini + Claude + 1 article for each concept

- Python data types in depth: Integer (whole numbers), Float (decimals), Boolean (True/False), String (text)
- Variables: naming conventions following PEP 8 standards (snake_case, descriptive names)
- Input/Output operations: print() function for output, input() function for user input
- Python syntax rules: indentation importance (4 spaces), case sensitivity, statement structure
- Comments: Single-line (#), Multi-line (''' '''), when and why to use them
- Understanding dynamic typing in Python: variables can change type
- Memory management basics: how Python stores different data types

Afternoon Session: Practical Implementation (4-5 hours)

- Create multiple Python scripts demonstrating each data type with examples
- Build interactive programs using input() and print() functions
- Practice proper variable naming following PEP 8 conventions
- Add comprehensive comments explaining each section of code
- Use type() function to check variable types at runtime
- Experiment with type conversion between different data types
- Create Jupyter notebook documenting all concepts with code examples

Practical Tasks

Task 1: Data Types Explorer

- Create script showing examples of int, float, bool, str
- Use type() function to display the type of each variable
- Show type conversion: int to str, str to float, float to int
- Print examples of each type with clear labels and formatting
- Add single-line comments explaining each variable
- Add multi-line docstring at top explaining script purpose

Task 2: Interactive Calculator (Basic)

- Prompt user for two numbers using input()
- Store in well-named variables (first_number, second_number - not x, y)
- Convert string inputs to appropriate numeric types (int or float)
- Calculate and display: sum, difference, product, quotient
- Format output clearly: 'The sum of 5 and 3 is: 8'
- Add error handling for invalid inputs (try-except)

Task 3: Research Task (MANDATORY)

- Research 'Python memory management and data types' from 3 AI sources
- Ask ChatGPT: 'How does Python handle memory for different data types?'
- Ask Gemini: 'Explain mutable vs immutable types in Python with examples'
- Ask Claude: 'What are best practices for naming variables in Python?'
- Read one article on 'Python data types and memory management'
- Document all sources with key takeaways in notebook References section

Expected Deliverables

- Two working Python scripts (.py files) with proper PEP 8 formatting
- Jupyter notebook with research notes, examples, and comparisons
- GitHub commit with descriptive message: 'Day 2: Python data types and variables'
- Updated LEARNINGS.md with insights from research
- Email report including one professional coding standard learned

Soft Skill Focus: Code clarity: Practice writing self-documenting code with descriptive variable names

Day 3: Conditional Statements & Logic

Morning Session: Theoretical Concepts (2-3 hours)

Research Requirements: Consult ChatGPT + Gemini + Claude + 1 article for each concept

- Conditional statements: if, elif, else syntax and structure
- Boolean logic and truth values: True, False, truthy and falsy values
- Comparison operators in detail: == (equals), != (not equals), >, <, >=, <=
- Practical use cases: user authentication, data validation, program flow control
- Flow control: how conditionals direct program execution path
- Nested if statements: when and how to use them properly
- Common pitfalls: assignment (=) vs comparison (==), indentation errors

Afternoon Session: Practical Implementation (4-5 hours)

- Build age verification program with multiple age categories
- Create number classification system (positive, negative, zero)
- Implement grade calculator with letter grades and messages
- Practice nested if statements for complex decision-making
- Add error handling for invalid inputs using conditionals
- Test edge cases: boundary values, unexpected inputs

Practical Tasks

Task 1: Enhanced Age Verification System

- Prompt user for name and age (use appropriate variable names)
- Classify into categories: Child (<13), Teenager (13-17), Adult (18-64), Senior (65+)
- Print personalized message for each category
- Example: 'Hello John! As a teenager, you have many opportunities ahead.'
- Handle invalid inputs: negative ages, non-numeric input
- Add comments explaining the logic of each condition

Task 2: Grade Calculator with Feedback

- Accept numeric grade (0-100) from user
- Assign letter grades: A (90-100), B (80-89), C (70-79), D (60-69), F (<60)
- Add personalized messages: A: 'Excellent work!', B: 'Good job!', etc.
- Validate input: must be between 0 and 100

- Use elif statements efficiently (avoid nested ifs when unnecessary)
- Print grade with encouraging message and grade interpretation

Task 3: Research Task (MANDATORY)

- Research 'Boolean logic in programming' from ChatGPT, Gemini, Claude
- Read one article on 'Writing clean conditional statements in Python'
- Ask each AI: 'What are best practices for if-elif-else statements?'
- Compare different coding styles for conditionals
- Document which style is most readable and why

Expected Deliverables

- Two programs demonstrating conditional logic with proper error handling
- Jupyter notebook with flowcharts visualizing program logic
- GitHub commit with clear documentation and test cases
- Email report with learning reflection on logic and flow control

Soft Skill Focus: Problem-solving: Learn to break complex problems into simple conditional logic steps

Day 4: Operators & Type Conversion

Morning Session: Theoretical Concepts (2-3 hours)

Research Requirements: Consult ChatGPT + Gemini + Claude + 1 article for each concept

- Logical operators in Python: and, or, not - truth tables and practical usage
- Comparison (Relational) operators: ==, !=, >, <, >=, <= - when to use each
- Type conversion functions: int(), str(), float(), bool() - explicit conversion
- Arithmetic operators: +, -, *, /, // (floor division), % (modulus), ** (exponentiation)
- Operator precedence: understanding order of operations in Python
- Common type conversion errors and how to handle them
- Real-world use cases: building calculators, form validation, data processing

Afternoon Session: Practical Implementation (4-5 hours)

- Build programs using multiple operators in combination
- Create login systems using logical operators for validation
- Practice type conversion in real-world scenarios (user input processing)
- Implement calculators with all arithmetic operations
- Test operator precedence with complex expressions
- Handle type conversion errors gracefully with try-except blocks

Practical Tasks

Task 1: Advanced Login System

- Create a login system that checks username, password, and age , Use logical operators (and, or, not) to validate: Username must be at least 5 characters Password must be at least 8 characters , Age must be 18 or above .Grant or deny access based on all conditions being met .
- Print specific error messages for each failed condition ,
- Add comments explaining the logical flow

Task 2: Multi-Operation Calculator

- Prompt user for two numbers and an operation (+, -, *, /, %, **)
- Convert string inputs to appropriate numeric types using type conversion
- Perform the selected operation and display the result
- Handle division by zero error
- Handle invalid operation selection
- Format output with proper decimal places for float results
- Example output: "5.0 + 3.0 = 8.0"

Task 3: Research Task (MANDATORY)

- Research 'Python operators and operator precedence' from 3 AI sources
- Ask ChatGPT: 'Explain logical operators with real-world examples'
- Ask Gemini: 'What is operator precedence in Python and why does it matter?'
- Ask Claude: 'Best practices for type conversion and error handling in Python'
- Read one article on 'Type conversion best practices in Python'
- Document comparison of all sources in notebook

Expected Deliverables

- Two functional programs demonstrating operators and type conversion
- Jupyter notebook with operator precedence examples and explanations
- GitHub commit: 'Day 4: Operators, type conversion, and login system'
- LEARNINGS.md updated with operator precedence insights
- Email report highlighting one challenge overcome today

Soft Skill Focus: Debugging - Practice identifying and fixing type-related errors systematically

Day 5 : Introduction to ML Concepts - Supervised / unsupervised / Reinforcement learning

Morning Session: Theoretical Concepts (2-3 hours)

Research Requirements: Consult ChatGPT + Gemini + Claude + 1 article for each concept

- Supervised Learning: Learning from labeled data (input-output pairs)
- Definition and real-world examples (email spam detection, house price prediction)
- Training process: model learns patterns from labeled examples

- Unsupervised Learning: Finding patterns in unlabeled data
- Definition and use cases (customer segmentation, anomaly detection)
- Key difference from supervised learning
- Reinforcement Learning: Learning through trial and error with rewards
- Definition and examples (game AI, robotics, recommendation systems)
- Agent, environment, actions, rewards concept
- Regression vs. Classification:
- Regression: Predicting continuous values (prices, temperatures, sales)
- Classification: Predicting discrete categories (spam/not spam, disease/healthy)
- Real-world examples of each
- Decision Tree Algorithm:
- How decision trees make decisions (branching logic)
- Tree structure: root node, internal nodes, leaf nodes
- Practical use cases in classification and regression

Afternoon Session: Practical Implementation (4-5 hours)

- Create visual flowcharts explaining ML concepts
- Build simple decision-making programs simulating decision trees
- Write Python programs demonstrating regression vs classification logic
- Research and document real-world ML applications
- Create comparison tables for different ML types

Practical Tasks

Task 1: ML Concepts Documentation

- Create a comprehensive Jupyter notebook explaining:
- Supervised, unsupervised, and reinforcement learning with examples
- Regression vs classification with 3 examples each
- Decision tree concept with a hand-drawn diagram
- Include code comments and markdown explanations
- Add visual diagrams or flowcharts for each concept

Task 2: Simple Decision Tree Simulator

- Build a Python program that simulates a decision tree for loan approval:
- Input: age, income, credit_score
- Decision rules:
  If age < 18: reject
  Else if income < 30000: reject
  Else if credit_score < 600: reject
  Else: approve
- Print the decision path taken
- Add comments explaining how this mimics a decision tree structure

Task 3: Communication Task

- Create a 1-minute video or voice recording explaining
- The difference between regression and classification
- Use real-world analogy (e.g., predicting house prices vs. identifying spam emails)
- Upload to Google Drive or record audio file
- Include transcript in your notebook

Task 4: Research Task (MANDATORY)

- Research 'Types of Machine Learning' from ChatGPT, Gemini, Claude
- Ask each: 'Explain supervised, unsupervised, and reinforcement learning with examples'
- Read one article: 'Introduction to Machine Learning for Beginners'
- Research 'Decision Trees in Machine Learning' from all sources
- Document comparison table showing which source explained each concept best

Expected Deliverables

- Jupyter notebook with ML concepts explained with diagrams
- Python script simulating decision tree logic
- Video/audio recording explaining regression vs classification (with transcript)
- GitHub commit: 'Day 5: Introduction to Machine Learning concepts'
- Updated LEARNINGS.md with ML insights from 4 sources
- Email report with reflection on ML understanding

Soft Skill Focus : Communication - Practice explaining complex technical concepts in simple terms

Day 6 : Python Data Structures - Lists

Morning Session: Theoretical Concepts (2-3 hours)

Research Requirements: Consult ChatGPT + Gemini + Claude + 1 article for each concept

- Lists in Python: Ordered, mutable collections that can hold multiple items
  - List syntax: my_list = [1, 2, 3, 'text', True]
  - Mixed data types allowed in lists
  - Indexing: accessing elements by position (0-indexed)
  - Negative indexing: accessing from the end (-1, -2, etc.)
- Accessing List Items:
  - Single item access: my_list[0]
  - Range access with slicing: my_list[1:4]
  - Step slicing: my_list[::2] (every second item)
- Adding Items to Lists:
  - append(): Add single item to end
  - insert(): Add item at specific position
  - extend(): Add multiple items from another list
- Removing Items from Lists:
  - remove(): Remove first occurrence of value

    - pop(): Remove and return item at index (default last)
    - clear(): Remove all items
- List Methods:
    - sort(): Sort list in place
    - reverse(): Reverse list in place
    - count(): Count occurrences of value
    - index(): Find index of first occurrence

Afternoon Session: Practical Implementation (4-5 hours)

- Create programs using various list operations
- Build student management systems using lists
- Practice list slicing with different patterns
- Implement sorting and searching in lists
- Handle list errors (IndexError) gracefully

Practical Tasks

Task 1: Student Management System

- Create a list of student names (at least 5 students)
- Add 2 new students using append() and insert()
- Remove 1 student using remove() and 1 using pop()
- Display the list after each operation with clear labels
- Create a sliced list showing only first 3 students
- Sort the list alphabetically and display
- Add comments explaining each operation

Task 2: Grade Tracker with Lists

- Create two parallel lists: student_names and student_grades
- Add at least 5 students with their grades
- Find and display:
  Highest grade and student name
  Lowest grade and student name
  Average grade
  Students who passed (grade >= 50)
- Use list methods and indexing appropriately
- Format output in a readable way

Task 3: List Slicing Practice

- Create a list of numbers from 1 to 20
- Demonstrate slicing:
  First 5 elements
  Last 5 elements

Every 3rd element
Reverse the entire list using slicing
Middle 10 elements

- Print each slice with explanation comments

Task 4: Research Task (MANDATORY)

- Research 'Python Lists and List Methods' from ChatGPT, Gemini, Claude
- Ask each: 'Explain list operations with practical examples'
- Read one article: 'Python Lists: A Complete Guide for Beginners'
- Ask: 'When should I use append() vs insert() vs extend()?'
- Document performance considerations for large lists
- Create comparison table of all list methods learned

Expected Deliverables

- Three Python scripts demonstrating list operations
- Jupyter notebook with list method examples and outputs
- GitHub commit: 'Day 6: Python lists and list manipulation'
- Updated LEARNINGS.md with list operations insights
- Email report with one challenge faced while working with lists

Soft Skill Focus: Attention to detail - Practice careful indexing and avoiding off-by-one errors

Day 7: Week 1 Assessment & Presentation

Morning Session: Presentation Preparation (2 hours)

Task 1: Prepare 10-Minute Technical Presentation

Create slides or live demo covering:

- Overview of Week 1 learning journey (30 seconds)
- Deep dive into your best project (Days 4-5) with live code walkthrough (5 minutes)
- Explanation of your research process: how you compared AI sources, synthesized information
- Challenges overcome: specific bug or concept that was difficult, how you resolved it
- Key takeaway for next week: one major insight about learning or professional development

Afternoon Session: Presentation & Feedback (3 hours)

Assessment Format

- 10-minute presentation to lead engineer or mentor

- 5-minute Q&A: technical questions about your code, design decisions, future improvements
- Code review session: lead reviews one of your scripts, provides feedback
- Discussion of research methodology: how effectively you used multiple sources

Post-Presentation Tasks

- Document all feedback received in FEEDBACK_WEEK1.md
- Create action items for Week 2 based on feedback
- Update GitHub with final Week 1 commits and presentation materials

Week 2 Preview: Python Data Structures & Algorithms

This week shifts focus from basics to data manipulation and algorithm thinking. You'll build real-world systems like student management, inventory tracking, and data analyzers using lists, dictionaries, and efficient loops.

Day 8: Lists & List Operations

Morning Session: Theoretical Concepts (2 hours)

Research Requirements

Consult ChatGPT + Gemini + Claude + 1 article for each concept:

- Lists in Python: ordered, mutable collections that can store any data type
- List methods: append(), insert(), remove(), pop(), sort(), reverse(), clear()
- List slicing: accessing sublists with [start:end:step], negative indexing
- List comprehensions: concise syntax for creating lists [x for x in range(10)]
- When to use lists: sequential data, ordered collections, frequent access by index

Afternoon Session: Practical Implementation (4 hours)

Task 1: Student Grade Manager

- Create list of student names and parallel list of their grades
- Add functions: add_student(), remove_student(), update_grade(), get_average()
- Find highest/lowest grades and corresponding students
- Sort students by grade (descending), display top 3 performers
- Use list comprehension to filter students above/below average

Task 2: Shopping Cart System

- Build shopping cart with items (name, price, quantity) stored as lists
- Functions: add_item(), remove_item(), update_quantity(), calculate_total()
- Apply discount if total > $100 (10% off)
- Display itemized receipt with item name, quantity, price, subtotal
- Use slicing to show 'recently added items' (last 3 items)

Task 3: Data Cleaning Pipeline

- Create list with messy data: duplicates, None values, extra spaces, mixed case
- Clean data: remove duplicates, filter out None, strip whitespace, normalize case
- Use list comprehension for each cleaning step
- Display before/after comparison with data quality metrics (unique count, completeness)

Day 9: Tuples & Sets

Morning Session: Theoretical Concepts (2 hours)

Research Requirements

Consult ChatGPT + Gemini + Claude + 1 article for each concept:

- Tuples: immutable sequences, faster than lists, used for fixed collections
- Tuple packing/unpacking: a, b = (1, 2), returning multiple values from functions
- Sets: unordered collections of unique elements, no duplicates allowed
- Set operations: union(), intersection(), difference(), symmetric_difference()
- When to use tuples vs lists vs sets: immutability needs, uniqueness requirements, lookup speed

Afternoon Session: Practical Implementation (4 hours)

Task 1: Geographic Coordinates System

- Store city locations as tuples: (city_name, latitude, longitude)
- Create function to calculate distance between two coordinate tuples
- Find closest city to a given coordinate, return tuple of (city, distance)
- Demonstrate tuple immutability: show error when trying to modify

Task 2: Unique Visitor Tracker

- Track website visitors using sets (automatically removes duplicate IPs)
- Compare visitors across different days: find common visitors (intersection)
- Find unique visitors for each day (difference), total unique visitors (union)
- Calculate visitor growth rate and retention rate

Task 3: Email Validation System

- Maintain set of valid email domains (gmail.com, yahoo.com, etc.)
- Validate emails: check if domain is in valid set, check for @ symbol
- Track unique email addresses registered (use set to prevent duplicates)
- Find emails from specific domains using set operations

Day 10: Dictionaries & JSON

Morning Session: Theoretical Concepts (2 hours)

Research Requirements

Consult ChatGPT + Gemini + Claude + 1 article:

- Dictionaries: key-value pairs, fast O(1) lookup, mutable
- Dictionary methods: get(), keys(), values(), items(), update(), pop()
- Nested dictionaries: storing complex structured data
- JSON format: reading/writing with json.load(), json.dump()
- Dictionary comprehensions: {k: v for k, v in pairs}

Afternoon Session: Practical Implementation (4 hours)

Task 1: Student Information System

- Create nested dictionary: {student_id: {name, age, grades{subject: score}}}
- Functions: add_student(), update_grade(), get_student_average(), find_top_student()
- Save/load student data to JSON file for persistence
- Generate report: all students with GPA, sorted by performance

Task 2: Product Inventory Manager

- Dictionary structure: {product_id: {name, price, quantity, category}}
- Functions: add_product(), update_stock(), search_by_category(), low_stock_alert()
- Calculate total inventory value, average product price per category
- Export inventory report as JSON

Task 3: Configuration Manager

- Create config.json with app settings (database, API keys, features)
- Build functions to load config, validate required keys, provide defaults
- Update config programmatically and save changes
- Handle missing keys gracefully with .get() method and default values

Day 11: Loops & Iteration

Morning Session: Theoretical Concepts (1.5 hours)

Research Requirements

Consult ChatGPT + Gemini + Claude + 1 article:

- For loops: iterating over sequences, range() function, enumerate(), zip()
- While loops: condition-based iteration, infinite loops, loop control
- Break, continue, else with loops: controlling loop flow, cleanup actions
- Nested loops: matrix operations, 2D data processing, time complexity $O(n^2)$

Afternoon Session: Practical Implementation (4.5 hours)

Task 1: Data Processing Pipeline

- Process list of 1000 records with for loop, apply transformations
- Use enumerate() to track progress (record #X of Y processed)
- Skip invalid records with continue, stop on critical error with break
- Use zip() to combine multiple data sources, validate matching lengths

Task 2: Pattern Generators

- Generate multiplication table (nested loops), pyramid patterns, number triangles
- Build matrix operations: transpose, sum rows/columns, find diagonal elements
- Create ASCII art generator using nested loops and conditionals

Task 3: Number Analysis System

- Find prime numbers up to N using while loop and mathematical optimization
- Calculate factorial, Fibonacci sequence using iterative approach
- Implement number guessing game with attempt limit and hint system

Day 12: Functions Fundamentals

Morning Session: Theoretical Concepts (1.5 hours)

Research Requirements

Consult ChatGPT + Gemini + Claude + 1 article:

- Function definition: def keyword, parameters, return values, None return
- Arguments: positional, keyword, default parameters, argument unpacking
- Scope: local vs global variables, global keyword, variable lifetime
- Function design principles: single responsibility, descriptive names, docstrings

Afternoon Session: Practical Implementation (4.5 hours)

Task 1: Math Utility Library

- Create functions: calculate_average(), find_median(), get_standard_deviation()
- Add parameter validation: check for empty lists, non-numeric values
- Write comprehensive docstrings with parameter types, return values, examples
- Use default parameters for optional settings (e.g., rounding precision)

Task 2: Text Processing Functions

- Build: count_words(), extract_emails(), remove_punctuation(), title_case()
- Chain functions: process_text(text, remove_punct=True, to_title=True)
- Return multiple values using tuples: word_count, char_count, unique_words

Task 3: Validation Function Suite

- Create: validate_email(), validate_phone(), validate_date(), validate_password()
- Each returns (is_valid: bool, error_message: str) tuple
- Build wrapper: validate_user_input() that calls appropriate validator

Day 13: Advanced Functions

Morning Session: Theoretical Concepts (1.5 hours)

Research Requirements

Consult ChatGPT + Gemini + Claude + 1 article:

- *args and **kwargs: variable arguments, when to use each, unpacking
- Lambda functions: anonymous functions, use in map/filter/sorted
- List comprehensions: [expr for item in iterable if condition]
- Dictionary comprehensions: {k: v for k, v in pairs if condition}

Afternoon Session: Practical Implementation (4.5 hours)

Task 1: Flexible Logger Function

- Create log(level, *messages, **options) accepting any number of messages
- Options: timestamp=True, file='log.txt', format='json'
- Format output based on level (ERROR: red, WARNING: yellow, INFO: normal)

Task 2: Data Transformer Suite

- Use lambda with map(): convert list of strings to uppercase, strip whitespace
- Use lambda with filter(): extract emails, phone numbers, URLs from text list
- Use lambda with sorted(): sort list of dicts by multiple keys
- Compare performance: lambda vs regular function vs list comprehension

Task 3: Advanced List/Dict Comprehensions

- Flatten nested lists: [item for sublist in nested for item in sublist]
- Matrix transpose using nested comprehension
- Dictionary inversion: {v: k for k, v in dict.items()}
- Build word frequency counter using dict comprehension

Day 14: Week 2 Review & Mini-Project

Morning Session: Comprehensive Review (2 hours)

Task 1: Data Structures Cheat Sheet

- Create comparison table: List vs Tuple vs Set vs Dict (mutability, ordering, use cases)
- Document time complexity: access, search, insertion, deletion for each
- Include code examples demonstrating when to use each structure

Afternoon Session: Mini-Project (4 hours)

Task 2: Contact Management System

Build a complete contact manager combining all Week 2 concepts:

- Data structure: dict of contacts {id: {name, phone, email, tags: set, notes: list}}
- Functions: add_contact(), search_contacts(), update_contact(), delete_contact()
- Advanced search: by name, tag, partial match using comprehensions
- Tag management: add_tag(), remove_tag(), find_by_tag() using set operations
- Export/import: save to JSON, load from JSON with error handling
- Statistics: total contacts, most used tags, contacts per category
- Interactive menu: loop-based CLI with numbered options

Evening: Presentation Prep

Prepare 5-minute demo of your contact manager showing:
- Key features and data structures used
- Live demonstration of add, search, tag operations
- Challenges faced and solutions implemented
- Code walkthrough: highlight use of comprehensions, functions, proper structure

Week 2 Key Takeaways

You've mastered Python's core data structures and can now build real data management systems. Week 3 transitions to AI and LangChain - you'll use these Python skills to build intelligent applications with language models.

Day 15: Introduction to Large Language Models

Morning Session: LLM Fundamentals (3 hours)

Research Requirements

Consult ChatGPT + Gemini + Claude + 2 technical articles:
- What are LLMs: transformer architecture basics, attention mechanism, pre-training vs fine-tuning
- Major LLM providers: OpenAI (GPT-4), Anthropic (Claude), Google (Gemini), model comparison
- Key concepts: tokens (input/output units), context window (memory limit), temperature (creativity)
- API parameters: max_tokens, temperature, top_p, frequency_penalty, presence_penalty
- Use cases: text generation, summarization, classification, Q&A, code generation, translation
- Limitations: hallucinations, knowledge cutoff, context limits, reasoning errors

Afternoon Session: API Integration (3-4 hours)

Task 1: OpenAI API Setup & Basic Calls

- Get API key from OpenAI platform, set up environment variable

- Install: pip install openai python-dotenv --break-system-packages
- Create .env file with API_KEY, add to .gitignore (NEVER commit keys!)
- Write first completion: simple prompt, print response, observe token usage
- Experiment with temperature: 0.0 (deterministic), 0.7 (balanced), 1.5 (creative)

Task 2: Parameter Exploration Notebook

- Test same prompt with different temperatures, compare outputs
- Experiment with max_tokens: observe truncation at different limits
- Try top_p sampling: 0.1 (focused), 0.9 (diverse), document differences
- Build comparison table: parameter → effect on output → best use case

Task 3: Build Simple Chatbot

- Create interactive loop: user input → API call → display response → repeat
- Add system message to define chatbot personality/role
- Implement conversation history: maintain context across turns
- Add error handling: API failures, rate limits, invalid responses
- Track costs: monitor token usage, calculate approximate cost per conversation

Day 16: LangChain Setup & First Chains

Morning Session: LangChain Concepts (2.5 hours)

Research Requirements

Consult ChatGPT + Gemini + Claude + 2 articles:

- What is LangChain: framework for LLM applications, abstracts API complexity
- Core components: Models (LLMs), Prompts (templates), Chains (workflows), Memory
- LCEL (LangChain Expression Language): | operator for chaining, runnable interface
- Document loaders: read PDFs, websites, databases into standardized format
- When to use LangChain vs raw APIs: complex workflows, document processing, agents

Afternoon Session: Hands-On Implementation (3.5 hours)

Task 1: LangChain Installation & Setup

- Install: pip install langchain langchain-openai langchain-community --break-system-packages
- Set up ChatOpenAI model wrapper, test with simple completion
- Create PromptTemplate: define variables, format strings, reusable templates
- Build first chain: prompt | model | output parser using LCEL

Task 2: Document Loaders Practice

- Load text file: TextLoader, examine Document structure (page_content, metadata)
- Load PDF: PyPDFLoader, handle multi-page documents
- Load webpage: WebBaseLoader, extract clean text from HTML
- Load CSV: CSVLoader, convert structured data to documents
- Build generic loader function: detect file type, use appropriate loader

Task 3: Build Document Q&A Chain

- Load document (any format), pass full content to LLM
- Create chain: load_doc → format_prompt → model → parse_answer
- Test with questions: 'Summarize this document', 'What are the key points?'
- Handle long documents: truncate or warn about context limits

Day 17: Text Embeddings & Semantic Search

Morning Session: Embedding Theory (3 hours)

Research Requirements

Consult ChatGPT + Gemini + Claude + 2 technical articles:

- What are embeddings: vector representations of text, semantic meaning in numbers
- How embeddings work: neural network encoding, high-dimensional space, similar texts → similar vectors
- Embedding models: OpenAI text-embedding-3-small/large, dimensions (512, 1536, 3072)
- Similarity metrics: cosine similarity, dot product, Euclidean distance
- Use cases: semantic search, clustering, recommendation, duplicate detection
- Vector databases: why traditional databases can't do semantic search efficiently

Afternoon Session: Practical Implementation (3 hours)

Task 1: Generate & Compare Embeddings

- Install: pip install openai numpy scikit-learn --break-system-packages
- Generate embeddings for sample sentences using OpenAI API
- Implement cosine_similarity() function from scratch
- Compare similar sentences: 'dog' vs 'puppy' → high similarity
- Compare different sentences: 'dog' vs 'car' → low similarity
- Visualize: create similarity matrix for 10 sentences, show heatmap

Task 2: Build Simple Semantic Search

- Create knowledge base: 50-100 sentences on various topics
- Embed all sentences, store in list with original text
- Search function: embed query, compute similarity to all stored embeddings
- Return top K most similar results with similarity scores

- Test queries: 'machine learning algorithms', 'healthy food recipes'
- Measure search quality: are results semantically relevant?

Task 3: Document Similarity Finder

- Load 10-20 documents (news articles, blog posts, etc.)
- Generate embedding for each full document
- Build clustering: group similar documents using similarity threshold
- Find duplicates: detect near-duplicate content (similarity > 0.95)
- Visualization: plot documents in 2D using dimensionality reduction (t-SNE or UMAP)

Day 18: Text Splitters & Chunking Strategies

Morning Session: Chunking Theory (2.5 hours)

Research Requirements

Consult ChatGPT + Gemini + Claude + 2 articles:

- Why chunk documents: LLM context limits, embedding model limits, retrieval precision
- Chunking strategies: fixed size, sentence-based, semantic, recursive character
- Chunk size considerations: 256-1024 tokens typical, overlap for context preservation
- LangChain splitters: RecursiveCharacterTextSplitter, TokenTextSplitter, MarkdownHeaderTextSplitter
- Metadata preservation: keep source, page number, section headers with each chunk
- Trade-offs: small chunks (precise, less context) vs large chunks (more context, less precise)

Afternoon Session: Implementation & Testing (3.5 hours)

Task 1: Compare Chunking Methods

- Load long document (research paper, article, book chapter)
- Split with CharacterTextSplitter: fixed 500 chars, no overlap
- Split with RecursiveCharacterTextSplitter: 500 chars, 50 overlap
- Compare results: chunk boundaries, context preservation, semantic coherence
- Document findings: which method preserves meaning better?

Task 2: Optimal Chunk Size Experiment

- Test same document with chunk sizes: 200, 500, 1000, 2000 characters
- Embed all chunks for each size, measure storage requirements
- Test retrieval: ask questions, see which chunk size returns best context
- Create report: chunk_size → avg_chunks → retrieval_quality → recommendation

Task 3: Smart Document Processor

- Build function: auto-detect document type (plain text, markdown, code)
- Apply appropriate splitter: MarkdownHeaderTextSplitter for .md, PythonCodeTextSplitter for .py
- Preserve structure: keep headers, code function definitions as metadata
- Add overlap intelligently: more overlap for technical content, less for narrative
- Output: list of chunks with rich metadata (source, type, section, tokens)

Day 19: Prompt Engineering Mastery

Morning Session: Advanced Prompting (3 hours)

Research Requirements

Consult ChatGPT + Gemini + Claude + 3 prompt engineering articles:

- Zero-shot prompting: task description only, no examples, suitable for simple tasks
- Few-shot prompting: provide 2-5 examples, dramatically improves accuracy
- Chain-of-Thought (CoT): 'Let's think step by step', improves reasoning tasks
- System messages: set behavior, role, constraints, output format expectations
- Prompt patterns: persona, instruction, context, format, examples, constraints
- Common pitfalls: vague instructions, assuming knowledge, no output format, conflicting instructions

Afternoon Session: Practical Experiments (3 hours)

Task 1: Prompting Technique Comparison

- Choose task: sentiment analysis, entity extraction, or text classification
- Zero-shot: basic instruction, test on 20 samples, measure accuracy
- Few-shot: add 3-5 examples, test same samples, compare accuracy
- Chain-of-Thought: add reasoning steps, test complex cases
- Document results: technique → accuracy → speed → cost → best use case

Task 2: Build Prompt Template Library

- Create templates for common tasks: summarization, extraction, generation, analysis
- Each template: system message, user instruction format, output format specification
- Add variables: {text}, {format}, {constraints}, {examples}
- Test each template on various inputs, refine based on results
- Store in JSON: {task_name: {system: '', user: '', examples: []}}

Task 3: Advanced Output Control

- Build prompt for JSON output: specify exact schema, enforce structure
- Build prompt for markdown table: enforce column alignment, handle varying rows
- Build prompt for code generation: specify language, style, docstrings
- Test robustness: try edge cases, malformed inputs, stress test constraints

Day 20: Structured Outputs with Pydantic

Morning Session: Pydantic Fundamentals (2.5 hours)

Research Requirements

Consult ChatGPT + Gemini + Claude + 2 articles:

- What is Pydantic: data validation library, type checking, automatic parsing
- BaseModel: define classes with typed fields, automatic validation, JSON serialization
- Field types: str, int, List, Optional, Union, custom validators
- LangChain integration: with_structured_output(), OutputParser, JsonOutputParser
- Why structured outputs: reliability, type safety, easier integration, error handling

Afternoon Session: Building Parsers (3.5 hours)

Task 1: Pydantic Model Suite

- Install: pip install pydantic --break-system-packages
- Create Person model: name (str), age (int), email (str), hobbies (List[str])
- Create Product model: id, name, price (float), in_stock (bool), tags (List[str])
- Add validators: email format, age > 0, price >= 0
- Test validation: correct data passes, invalid data raises errors

Task 2: LLM → Structured Data Pipeline

- Define model: Article (title, author, published_date, summary, tags)
- Use with_structured_output(): chain = model.with_structured_output(Article)
- Feed raw article text, get validated Article object
- Handle errors: retry on validation failure, log errors, default values
- Process batch: 10-20 articles, convert to structured JSON dataset

Task 3: Multi-Entity Extraction System

- Define nested models: Company (name, location, Employee[])
- Employee model: name, title, department, skills (List[str])
- Extract from unstructured text: company descriptions, press releases
- Build knowledge graph: companies → employees → skills
- Export to JSON, validate all extracted data, calculate extraction accuracy

Day 21: Week 3 Presentations & Review

Morning Session: Project Consolidation (2 hours)

Task 1: Build Comprehensive Demo Application

Combine Week 3 concepts into one application:

- Document Analyzer: upload PDF/text, extract embeddings
- Smart chunking: use RecursiveCharacterTextSplitter with optimal size
- Semantic search: query documents, return relevant chunks
- Structured extraction: use Pydantic to extract entities from documents
- Output report: summarize findings, list entities, show relevance scores

Afternoon Session: Technical Presentation (4 hours)

Presentation Structure (15 minutes)

- Introduction (2 min): Week 3 overview, learning objectives achieved
- Architecture walkthrough (4 min): system diagram, component interaction
- Live demo (5 min): upload document, run search, show structured output
- Technical deep-dive (3 min): interesting problem solved, code snippet review
- Reflection (1 min): key learnings, challenges overcome, next steps

Q&A Session (10 minutes)

Be prepared to discuss:

- Why you chose specific chunking strategy and model parameters
- Trade-offs in your design decisions (e.g., chunk size, embedding model)
- How you would scale this system for production use
- Challenges with prompt engineering and how you addressed them

Post-Presentation Documentation

- Create comprehensive README: setup instructions, architecture, usage examples
- Document feedback received, create action items for Week 4
- Write reflection: What worked well? What would you do differently?

Week 4 Preview: RAG Systems & Vector Databases

Next week, you'll take your LangChain knowledge to production level. You'll integrate vector databases (FAISS, Pinecone), build sophisticated RAG pipelines, implement agent workflows, and create systems that combine retrieval with generation for accurate, contextual AI responses.

Day 22: Vector Stores & Databases

Morning Session: Vector Database Theory (3 hours)

Research Requirements

Consult ChatGPT + Gemini + Claude + 2 technical articles:

- What are vector databases: specialized for storing and querying high-dimensional vectors
- Why not traditional databases: similarity search requires approximate nearest neighbor (ANN) algorithms
- Indexing algorithms: HNSW (hierarchical navigable small world), IVF (inverted file index)
- Vector database options: FAISS (local, fast), Pinecone (cloud, managed), Qdrant (open-source, production)
- Key operations: add vectors, similarity search with K results, filter by metadata
- Performance metrics: recall (accuracy), latency (speed), throughput (queries/second)

Afternoon Session: Hands-On Implementation (3 hours)

Task 1: FAISS Setup & Basic Operations

- Install: pip install faiss-cpu langchain-community --break-system-packages
- Create vector store: load documents, generate embeddings, store in FAISS
- Implement add, search, delete operations
- Save/load index: persist to disk, reload for reuse
- Test similarity search: query → K nearest neighbors with scores

Task 2: Document Library with FAISS

- Build library: 50-100 documents on various topics
- Chunk documents optimally (recursive splitter, 800 chars, 100 overlap)
- Generate embeddings, store in FAISS with metadata (source, page, section)
- Build search interface: query → retrieve top 5 chunks → display with metadata
- Add metadata filtering: search within specific source or section

Task 3: Vector Store Comparison

- Install Chroma: pip install chromadb --break-system-packages
- Same dataset, test with FAISS vs Chroma
- Measure: indexing time, query latency, memory usage, result quality
- Document findings: FAISS (fast, local) vs Chroma (persistent, feature-rich)

Day 23: RAG Pipeline Development

Morning Session: RAG Architecture (2.5 hours)

Research Requirements

Consult ChatGPT + Gemini + Claude + 3 RAG articles:

- RAG workflow: Index (chunk + embed), Retrieve (search vectors), Generate (LLM with context)

- Why RAG: reduces hallucinations, grounds answers in documents, handles private data
- RAG vs fine-tuning: RAG for dynamic knowledge, fine-tuning for style/behavior
- LangChain RAG components: VectorStore, Retriever, RetrievalQA chain
- Quality challenges: retrieval precision, context relevance, answer accuracy

Afternoon Session: Build RAG System (3.5 hours)

Task 1: Simple RAG Pipeline

- Load documents, chunk, embed, store in FAISS
- Create retriever: as_retriever(search_kwargs={'k': 4})
- Build RetrievalQA chain: retriever + LLM + prompt template
- Test queries: ask questions, verify answers cite documents correctly
- Add source attribution: show which document chunks were used

Task 2: Enhanced RAG with Custom Prompts

- Design prompt template: 'Answer based ONLY on context. Context: {context} Question: {question}'
- Add instructions: 'If answer not in context, say "I don't have that information"'
- Implement chain: retriever → format_context → prompt → LLM → parse
- Test edge cases: queries with no relevant docs, ambiguous questions, follow-ups

Task 3: Multi-Document RAG System

- Index multiple document sources: PDFs, websites, text files
- Tag each with metadata: source_type, domain, date
- Implement filtered retrieval: search only specific source types
- Build answer synthesis: combine information from multiple retrieved chunks
- Output format: answer + list of sources with relevance scores

Day 24: Context Engineering & FastAPI Basics

Morning Session: Advanced Context Management (2 hours)

Research Requirements

Consult ChatGPT + Gemini + Claude + 2 articles:

- Context window limits: 4K, 8K, 128K tokens, staying within bounds
- Dynamic context injection: add relevant info at inference time
- Memory types: conversation buffer, summary memory, entity memory
- Context compression: summarize old messages, keep recent detailed

Task 1: Conversational RAG with Memory

- Add ConversationBufferMemory to RAG chain
- Maintain chat history: previous Q&A pairs available for context
- Handle follow-up questions: 'What about X?' without repeating full query

- Implement memory pruning: keep last 10 exchanges, summarize older

Afternoon Session: FastAPI Introduction (4 hours)

Research Requirements

Consult ChatGPT + Gemini + Claude + 2 FastAPI tutorials:

- What is FastAPI: modern Python web framework, async support, automatic API docs
- Key features: type hints, Pydantic validation, OpenAPI/Swagger UI auto-generation
- REST API concepts: GET, POST, PUT, DELETE, path parameters, query parameters, request body
- Why FastAPI for AI: async for long LLM calls, easy integration, production-ready

Task 2: First FastAPI Application

- Install: pip install fastapi uvicorn --break-system-packages
- Create main.py: FastAPI instance, define routes
- Build endpoints: GET /health, GET /items/{id}, POST /items
- Use Pydantic models for request/response validation
- Run server: uvicorn main:app --reload, test with browser and curl
- Explore auto-docs: visit /docs for Swagger UI, /redoc for ReDoc

Task 3: RAG API Endpoint

- Create POST /ask endpoint accepting question (str)
- Load RAG system on startup (not per request): use @app.on_event('startup')
- Process request: retrieve → generate → return {answer, sources, confidence}
- Add error handling: 404 for no results, 500 for LLM errors, 400 for invalid input
- Test with multiple clients: Postman, curl, Python requests library

Day 25: Production-Ready RAG & FastAPI Integration

Morning Session: Advanced RAG Techniques (3 hours)

Research Requirements

Consult ChatGPT + Gemini + Claude + 3 advanced RAG articles:

- Retrieval strategies: dense (embeddings), sparse (BM25), hybrid (combine both)
- Re-ranking: retrieve many candidates (20), re-rank top K (5) for quality
- Query expansion: reformulate query, generate multiple variations, retrieve for each
- Self-query: LLM extracts structured filters from natural language query
- Parent document retrieval: retrieve small chunks, return full parent document

Task 1: Implement Hybrid Search

- Install: pip install rank-bm25 --break-system-packages

- Build BM25 index: tokenize documents, create keyword-based retriever
- Implement ensemble retriever: combine FAISS (semantic) + BM25 (keyword)
- Weight results: 70% semantic, 30% keyword for balanced retrieval
- Test: compare pure semantic vs hybrid, measure recall improvement

Task 2: Add Re-ranking Layer

- Retrieve top 20 candidates from vector store
- Use LLM to score each candidate: relevance to query (0-10)
- Select top 5 highest-scored chunks for final context
- Measure impact: better answer quality, higher precision

Afternoon Session: Complete FastAPI RAG Service (3 hours)

Task 3: Build Production RAG API

Create comprehensive API with these endpoints:

- POST /documents/upload: accept file, chunk, embed, index
- GET /documents: list all indexed documents with metadata
- DELETE /documents/{id}: remove document from index
- POST /ask: query RAG system, return answer + sources
- POST /search: semantic search only (no generation), return chunks
- GET /health: system status, indexed document count, memory usage

Implementation Requirements:

- Async endpoints: async def for all routes that call LLM/embedding API
- Proper error handling: try/except, return appropriate HTTP status codes
- Request validation: Pydantic models for all input/output
- Logging: track all requests, errors, processing times
- CORS: enable cross-origin requests for frontend integration
- Environment variables: use .env for API keys, config

Day 26: LangChain Agents & Tool Calling

Morning Session: Agent Fundamentals (3 hours)

Research Requirements

Consult ChatGPT + Gemini + Claude + 3 agent framework articles:

- What are agents: LLMs that can use tools and make decisions autonomously
- ReAct framework: Reason (think), Act (use tool), Observe (see result), repeat
- Agent types: zero-shot, conversational, structured tool, OpenAI functions
- Tools: Python functions with descriptions, parameters, return values
- Agent loop: planning → tool selection → execution → result integration → next step
- Challenges: tool selection errors, infinite loops, cost management

Afternoon Session: Building Agents (3 hours)

Task 1: Create Custom Tools

- Build calculator tool: accepts math expression, returns result
- Build web search tool: takes query, returns summarized results
- Build RAG tool: searches your document index, returns relevant info
- Each tool: clear description, typed parameters, error handling
- Use @tool decorator for automatic integration

Task 2: Build ReAct Agent

- Initialize agent: create_react_agent(llm, tools, prompt)
- Test with queries requiring multiple tools: 'What's 23 * 45? Then search for info on that number'
- Add verbose mode: see agent's reasoning process, tool selections
- Handle failures: max iterations limit, invalid tool usage, retry logic

Task 3: Multi-Tool Research Assistant

- Create agent with: RAG search, web search, calculator, date/time tools
- Complex query: 'Find information about X in my docs, then search web for recent updates'
- Add memory: agent remembers conversation context, refers to previous findings
- Track agent performance: tool usage frequency, success rate, average cost

Day 27: Weekend Integration - Full-Stack RAG Application

Morning Session: Architecture Design (2 hours)

Task 1: System Design Document

Create comprehensive design for production RAG system:

- Architecture diagram: components (FastAPI, FAISS, LangChain, agents), data flow
- Database schema: documents, chunks, embeddings, conversations, user sessions
- API specification: all endpoints, request/response schemas, error codes
- Deployment plan: Docker containerization, environment management, monitoring

Afternoon Session: Implementation (6 hours)

Task 2: Backend Development

- Project structure: /app (main code), /tests, /docs, /data, docker-compose.yml
- Implement all API endpoints from Day 25 design
- Add PostgreSQL: store document metadata, user queries, system logs
- Install: pip install sqlalchemy psycopg2-binary --break-system-packages
- Database models: Document, Chunk, Conversation using SQLAlchemy ORM
- Add authentication: JWT tokens for API access (optional but recommended)

Task 3: Advanced Features

- Conversation threading: group related Q&A, maintain context per thread
- Document versioning: track updates, allow rollback
- Analytics endpoint: query volume, popular questions, average response time
- Export functionality: download conversation history, search results as JSON
- Rate limiting: prevent abuse, implement usage quotas

Task 4: Testing & Documentation

- Write unit tests: test each function, edge cases, error handling
- Integration tests: test full API workflows, database interactions
- Install: pip install pytest pytest-asyncio httpx --break-system-packages
- Create comprehensive README: setup, usage, API docs, examples
- API documentation: use FastAPI's auto-docs, add detailed descriptions

Day 28: Final Presentations & Portfolio Building

Morning Session: Presentation Preparation (3 hours)

Task 1: Create Technical Presentation (20 minutes)

Presentation Structure:

- Introduction (2 min): Problem statement, solution overview
- Architecture (4 min): System diagram, components, tech stack justification
- Live demo (8 min): Upload document → Ask questions → Show agent workflow → Display analytics
- Technical deep-dive (4 min): Interesting challenge solved, code walkthrough
- Results & impact (2 min): Performance metrics, potential use cases

Demo Preparation:

- Prepare test documents covering different domains
- Create script of questions showcasing different features
- Test demo flow multiple times, have backup plan
- Record demo video as backup in case of technical issues

Afternoon Session: Final Presentation (4 hours)

Assessment Format

- 20-minute technical presentation
- 10-minute Q&A: architecture decisions, scaling considerations, improvements
- Code review: walk through key components, explain design patterns
- Discussion: 30-day learning journey, growth areas, next steps

Expected Discussion Topics:

- Why you chose specific technologies (FAISS vs Pinecone, embedding models)
- How you would scale this to handle 1M documents and 1000 req/sec

- Security considerations: authentication, data privacy, API security
- Cost optimization: reducing API calls, caching strategies, batch processing
- Most challenging technical problem and how you solved it

Portfolio Development

Task 2: Professional GitHub Repository

- Comprehensive README: project overview, features, tech stack, setup guide
- Architecture diagram: system components, data flow, deployment
- API documentation: endpoint descriptions, request/response examples
- Screenshots/GIFs: show key features, user interface, results
- License, contributing guidelines, acknowledgments

Task 3: LinkedIn Post & Portfolio Update

- Write LinkedIn post: 30-day journey, key learnings, project showcase
- Include metrics: lines of code, documents processed, API endpoints built
- Tag Xeven Solutions, thank mentors, share GitHub link
- Update resume: add AI Engineer Intern experience, list technologies
- Portfolio website: add project card with description, demo link, code

Final Reflection & Next Steps

Reflection Essay (1000 words):

- Technical growth: most valuable skills learned, areas of confidence
- Professional development: research methodology, documentation, presentation skills
- Challenges overcome: toughest problems, debugging strategies, persistence
- Key insights: about AI engineering, LangChain ecosystem, production systems
- Career readiness: how prepared you feel, areas for continued learning
- Future plans: next projects, technologies to explore, career goals

Congratulations! 🎉

You've completed an intensive 30-day AI Engineering internship, building production-ready RAG systems with LangChain and FastAPI. You've mastered:

- Python fundamentals and data structures
- LLM integration and prompt engineering
- Vector databases and semantic search
- RAG pipeline development and optimization
- FastAPI for production APIs
- LangChain agents and tool calling
- Professional development: documentation, testing, presentations

You're now ready to contribute to real-world AI engineering projects. Keep building, keep learning, and welcome to the AI engineering community!

Appendix: Resources & Templates

Essential Learning Resources

- **Python Official Tutorial:** https://docs.python.org/3/tutorial/
- **PEP 8 Style Guide:** https://peps.python.org/pep-0008/
- **LangChain Documentation:** https://python.langchain.com/docs/
- **FastAPI Documentation:** https://fastapi.tiangolo.com/
- **PostgreSQL Tutorial:** https://www.postgresqltutorial.com/
- **GitHub Docs:** https://docs.github.com/en/get-started
- **Real Python Tutorials:** https://realpython.com/
- **Medium AI Articles:** https://medium.com/tag/artificial-intelligence

Email Progress Report Template

Subject: [Day X] [Your Name] - Daily Progress Report

Dear [Lead Name],

Here is my progress report for Day X of the AI Engineer Internship:

What I Learned (Theoretical Concepts):

• Concept 1: Brief explanation of what you learned

• Concept 2: Brief explanation with key insight

• Concept 3: How it connects to previous learning

What I Built (Practical Implementations):

• Project 1: Description, outcome, challenges overcome

• Project 2: Description, what works, what needs improvement

Research Sources Consulted:

1. ChatGPT (January X, 2026): Key takeaway - [summary]

2. Google Gemini (January X, 2026): Key takeaway - [summary]

3. Claude (January X, 2026): Key takeaway - [summary]

4. Article: "[Title]" by [Author] - URL: [link]

Key insight: [what you learned from article]

Comparison: Which source was clearest and why

Challenges & Solutions:

• Challenge: [Describe technical problem faced]

Solution: [How you solved it - steps taken, resources used]

Professional Insight:

• One soft skill or team practice learned today

• Example: "Learned importance of descriptive commit messages for collaboration"

GitHub Commit:

Repository: https://github.com/[username]/ai-internship-xeven-2026

Commit: [commit hash or message]

Direct link: [full URL to commit]

Questions/Blockers for Tomorrow:

• Question 1: [Specific technical question with context]

• Blocker: [If stuck, describe what you've tried]

Thank you for your guidance and support!

Best regards,

[Your Name]

AI Engineer Intern

Xeven Solutions

Success Criteria

- Complete all 30 days of practical tasks with working, well-documented code
- Maintain daily GitHub commits with proper PEP 8 compliance and descriptive messages
- Research documentation citing 3 AI sources + 1 article for EVERY concept (documented in notebooks)
- Send email progress reports daily to assigned lead with all required sections
- Participate in all weekly presentations (Week 1, 2, 3, 4) with live demos

- Build 3 major projects: RAG System, FastAPI Application, Final Capstone Integration
- Demonstrate professional soft skills: code reviews, documentation, communication, time management
- Maintain professional GitHub profile with green contribution graph throughout program
- Complete final presentation covering technical skills and professional growth
- Submit final reflection essay analyzing your 30-day journey and learnings

Note for Leads: Review each intern's progress daily through GitHub commits and email reports. Provide constructive feedback within 24 hours. Schedule regular check-ins to ensure understanding. Encourage collaboration while maintaining individual accountability. Celebrate small wins and provide support for challenges.